

CS 6083 – Principles of Database Systems · Spring 2026

Snickr: A Web-Based Team Collaboration System

Database Design and Web Frontend

Shyam Krishna Sateesh ss20355

<https://github.com/shyamsateesh/snickr>

<https://snickr-production.up.railway.app>

May 2026

Abstract

Snickr is a web-based team collaboration system built over two project phases for CS 6083. The system allows users to register, create workspaces and channels, invite other users, post and search messages, and manage administrative permissions. Project 1 produced a fully normalized nine-table PostgreSQL schema with an ER diagram, constraint definitions, and seven validated SQL queries. Project 2 added a Flask web frontend with complete CRUD functionality, session-based authentication, SQL injection and XSS protection, and three explicit database transactions. The application is publicly deployed at <https://snickr-production.up.railway.app>.

Contents

1	Introduction	3
2	ER Diagram	3
2.1	Strong Entity Sets and Attributes	3
2.2	Relationships	3
2.3	Total Participation Constraints	4
3	Relational Schema	5
3.1	Translation from ER to Relational Model	5
3.2	Relation Schemas	5
4	Design Decisions	6

5	Constraints	7
5.1	Primary Keys	7
5.2	Foreign Key Constraints	7
5.3	Uniqueness Constraints	8
5.4	Check Constraints	8
5.5	Default Values	8
6	SQL Queries	8
6.1	Query 1: Create a New User Account	8
6.2	Query 2: Create a New Public Channel (with Authorization Check) . . .	9
6.3	Query 3: List All Administrators per Workspace	9
6.4	Query 4: Public Channel Invitations Older Than 5 Days With No Join .	9
6.5	Query 5: All Messages in a Channel, Chronological Order	10
6.6	Query 6: All Messages Posted by a Particular User	10
6.7	Query 7: Accessible Messages Containing a Keyword	10
7	Sample Data	11
8	Assumptions	11
9	Project 2: Web Frontend	12
9.1	Overview	12
9.2	Technology Stack	12
9.3	System Architecture	12
9.4	Project Structure	12
9.5	Session Management	13
9.6	Routes	13
10	Security	14
10.1	SQL Injection Prevention	14
10.2	Cross-Site Scripting Prevention	14
10.3	Password Security	14
10.4	Authorization	14
11	Transactions	14
11.1	Create Workspace	15
11.2	Accept Workspace Invitation	15
11.3	Accept Channel Invitation	15
11.4	Concurrency	15
12	Application Walkthrough	15
13	Additional Features	16
14	Deployment	17

Introduction

Snickr is modelled after Slack: a platform where users create *workspaces* (top-level groups), communicate in typed *channels* within those workspaces, and exchange *messages* chronologically. Three channel types — public, private, and direct — provide graduated access control. Workspace administrators control membership; channel members control channel-level invitation.

The project proceeded in two phases. Phase 1 focused on database design: identifying entities and relationships, producing an ER diagram in crow's foot notation, translating to a relational schema, defining constraints, and writing and testing seven SQL queries against sample data. Phase 2 built a complete web application on top of the Phase 1 schema, implementing every user-facing feature through a Flask backend with Jinja2 templates and a single PostgreSQL connection.

This document covers both phases and serves as the complete project writeup. Phase 1 material has been revised to incorporate improvements made during Phase 2 development — specifically the addition of total participation constraints to the ER diagram and unique constraints on the invitation tables.

ER Diagram

The ER diagram was designed using crow's foot notation and captures four strong entity sets and nine relationship sets, including two ternary relationships for workspace and channel invitations.

Strong Entity Sets and Attributes

- **user**: user_id, email, username, nickname, password_hash, created_at
- **workspace**: workspace_id, name, description, created_at
- **channel**: channel_id, name, channel_type, created_at
- **message**: message_id, body, posted_at

No weak entity sets appear in the schema. Every entity has a surrogate **SERIAL** primary key and can be uniquely identified without referencing a parent entity. All stored fields are raw values; aggregations such as message counts are computed at query time.

Relationships

- **creates** (user $\rightarrow$ workspace): One-to-many. Each workspace has exactly one creator.
- **admin_of** (user $\leftrightarrow$ workspace): Many-to-many. Attribute: **granted_at**.
- **workspace_member** (user $\leftrightarrow$ workspace): Many-to-many. Attributes: **joined_at**, **is_active**.
- **workspace_invitation** (user $\times$ user $\times$ workspace): Ternary. Involves the user entity twice in the roles of *inviter* and *invitee*. Attributes: **status**, **invited_at**, **responded_at**.

- **contains** (workspace $\rightarrow$ channel): One-to-many. Each channel belongs to exactly one workspace.
- **channel_member** (user $\leftrightarrow$ channel): Many-to-many. Attribute: `joined_at`.
- **channel_invitation** (user $\times$ user $\times$ channel): Ternary. Same inviter/invitee role structure as workspace invitation.
- **posts** (user $\rightarrow$ message): One-to-many. Each message has exactly one sender.
- **belongs_to** (message $\rightarrow$ channel): Many-to-one. Each message belongs to exactly one channel.

Total Participation Constraints

The ER diagram was updated during Phase 2 to add total participation markers (double lines in crow's foot notation). The following entities exhibit total participation in the indicated relationships:

- **message** in *posts*: every message must have exactly one sender. Total participation on the message side.
- **message** in *belongs_to*: every message must belong to exactly one channel. Total participation on the message side.
- **channel** in *contains*: every channel must belong to exactly one workspace. Total participation on the channel side.
- **workspace** in *creates*: every workspace must have exactly one creator. Total participation on the workspace side.
- **workspace_invitation** in its connections to *inviter* and *invitee*: every invitation must reference exactly one inviter and one invitee.
- **channel_invitation** in its connections to *inviter* and *invitee*: same as above.

Entities on the optional sides — users in most relationships, workspaces and channels with respect to invitations — retain partial participation, since a user need not create any workspace or hold any admin role to exist in the system.

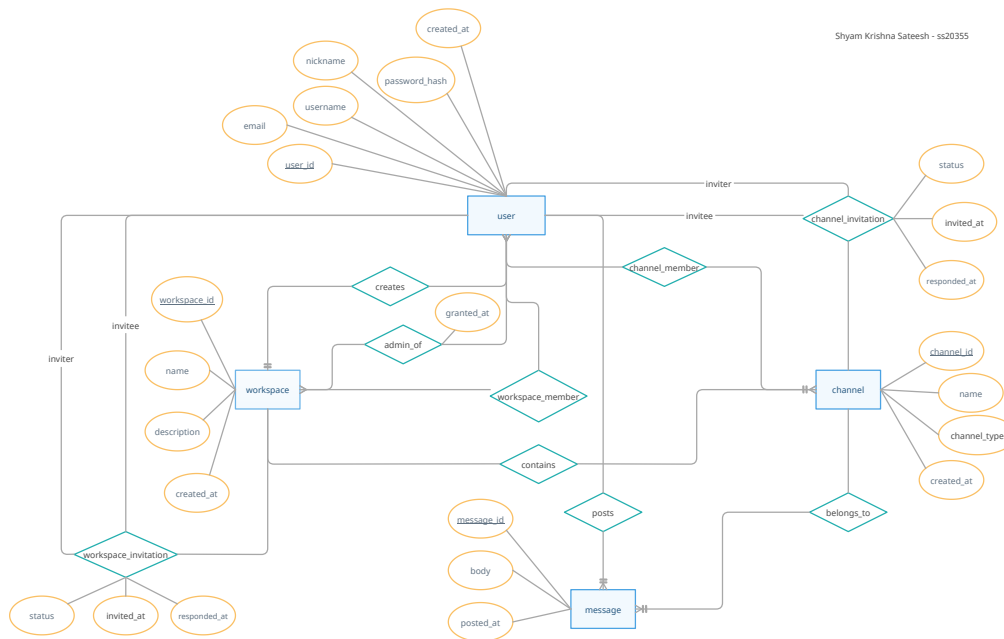


Figure 1: ER diagram for Snickr (crow's foot notation, with total participation constraints)

Relational Schema

Translation from ER to Relational Model

The ER-to-relational translation follows standard rules. One-to-many relationships (*creates*, *contains*, *posts*, *belongs_to*) are absorbed as foreign key columns into the many-side table. Many-to-many relationships (*admin_of*, *workspace_member*, *channel_member*) each become junction tables with composite primary keys. Ternary relationships (*workspace_invitation*, *channel_invitation*) each become their own table with a surrogate primary key and three foreign key columns.

This produces nine relation schemas. Primary keys are underlined; foreign keys are marked (FK).

Relation Schemas

- `users(user_id, email, username, nickname, password_hash, created_at)`
- `workspace(workspace_id, name, description, creator_id(FK), created_at)`
- `workspace_admin(workspace_id(FK), user_id(FK), granted_at)`

- `workspace_member(workspace_id(FK), user_id(FK), is_active, joined_at)`
- `workspace_invitation(invitation_id, workspace_id(FK), inviter_id(FK), invitee_id, status, invited_at, responded_at)`
- `channel(channel_id, workspace_id(FK), name, channel_type, creator_id(FK), created_at)`
- `channel_member(channel_id(FK), user_id(FK), joined_at)`
- `channel_invitation(invitation_id, channel_id(FK), inviter_id(FK), invitee_id(FK), status, invited_at, responded_at)`
- `message(message_id, channel_id(FK), sender_id(FK), body, posted_at)`

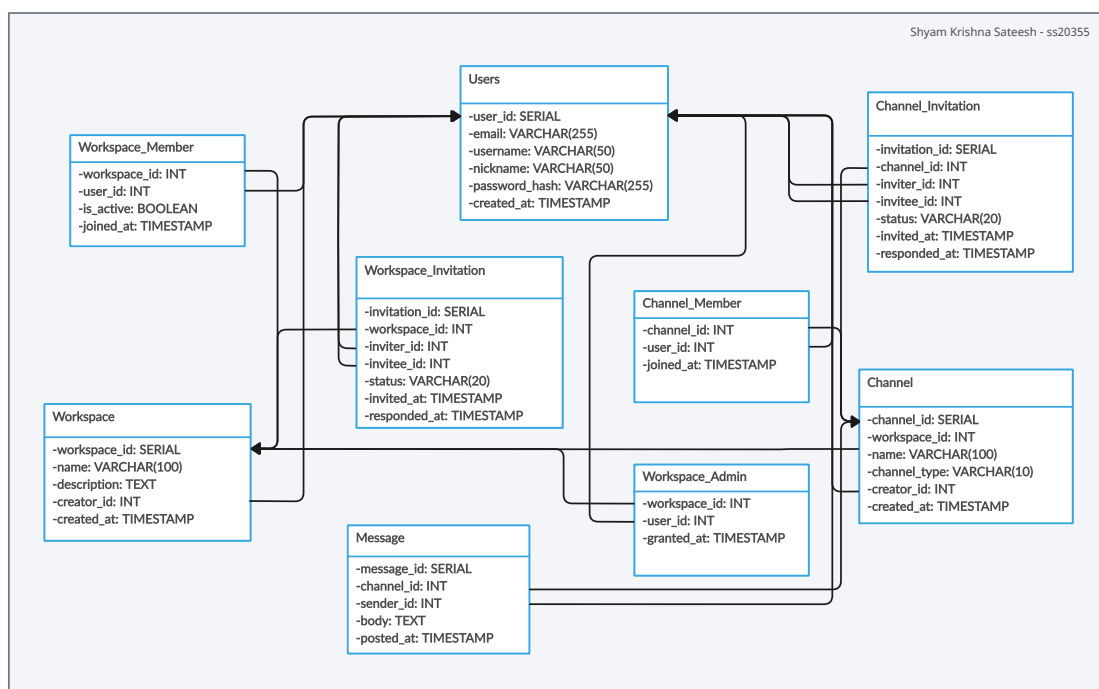


Figure 2: Relational schema diagram for Snickr

Design Decisions

Single channel table with channel_type. An alternative design would create three separate tables — `public_channel`, `private_channel`, `direct_channel`. A single table with a `channel_type` CHECK constraint is more space-efficient, eliminates schema duplication, and allows every channel query to target one table regardless of type.

Separate workspace_admin and workspace_member tables. Membership and administrator privilege are independent states. A user can be a member without being an admin, and admin rights can be granted or revoked without affecting membership. Merging into a single table with an `is_admin` column would also conflate the `granted_at` and `joined_at` timestamps, which track different events.

Surrogate SERIAL primary keys on all tables. Natural keys (such as username or email) can change. Surrogate integer keys are stable, compact for joins, and make every entity independently addressable without relying on composite natural identifiers.

Soft delete on workspace_member via is_active. Hard-deleting a workspace_member row would permanently remove the record of when the user joined, making historical queries impossible. The is_active flag supports the remove-member operation while preserving the row and its joined_at timestamp. All active-member queries include AND is_active = TRUE.

Application-level two-user constraint for direct channels. The spec defines direct channels as conversations between exactly two users. Enforcing this at the schema level would require a trigger. Enforcing it in application code is simpler and easier to maintain: when a direct channel is created, the second user is immediately sent a channel invitation, and no further members are added by any route.

Workspace creator inserted into both member and admin tables transactionally. The schema stores admin status in workspace_admin, not derivable from workspace.creator_id alone. When a workspace is created, the application inserts into workspace, workspace_member, and workspace_admin within a single transaction, guaranteeing the creator always holds both roles.

No message deletion. The spec explicitly states that deletion of messages by administrators is not required. No soft-delete flag is implemented on the message table, keeping the schema simple and avoiding cascade complexity.

Unique constraints on invitation tables. During Phase 2 development, a bug was identified where concurrent admin actions could insert duplicate invitation rows. Unique constraints were added to workspace_invitation(workspace_id, invitee_id) and channel_invitation(channel_id, invitee_id), and all insertion code uses ON CONFLICT DO NOTHING.

Constraints

Primary Keys

Every table has a designated primary key. workspace_admin, workspace_member, and channel_member use composite primary keys from two foreign key columns. Invitation tables use surrogate SERIAL keys to permit re-invitation after a declined invitation is superseded.

Foreign Key Constraints

All foreign key references are enforced at the database level:

- workspace.creator_id → users.user_id
- workspace_admin.{workspace_id, user_id} → workspace, users
- workspace_member.{workspace_id, user_id} → workspace, users
- workspace_invitation.{workspace_id, inviter_id, invitee_id} → workspace, users, users

- `channel.workspace_id` → `workspace.workspace_id`
- `channel.creator_id` → `users.user_id`
- `channel_member.{channel_id, user_id}` → `channel, users`
- `channel_invitation.{channel_id, inviter_id, invitee_id}` → `channel, users, users`
- `message.channel_id` → `channel.channel_id`
- `message.sender_id` → `users.user_id`

Uniqueness Constraints

- `users.email` — UNIQUE. No two accounts share an email address.
- `users.username` — UNIQUE. Usernames are globally distinct.
- `workspace_invitation(workspace_id, invitee_id)` — UNIQUE. Prevents duplicate pending invitations.
- `channel_invitation(channel_id, invitee_id)` — UNIQUE. Same rationale.

Check Constraints

- `channel.channel_type` must be one of 'public', 'private', or 'direct'.
- `workspace_invitation.status` and `channel_invitation.status` must each be one of 'pending', 'accepted', or 'declined'.

Default Values

All `created_at`, `joined_at`, `granted_at`, `invited_at`, and `posted_at` columns default to `NOW()`. `workspace_member.is_active` defaults to `TRUE`. Invitation status defaults to 'pending'.

SQL Queries

Seven queries were written and tested against the sample data for Project 1. In Project 2 these queries are re-used in `db.py` with `%s` placeholders replacing hardcoded values, which is how `psycopg2` parameterization works.

Query 1: Create a New User Account

```
1 INSERT INTO users (email, username, nickname, password_hash)
2 VALUES ('eve@nyu.edu', 'eve', 'Evie', 'hash5');
```

Result: INSERT 0 1. In the web application, `werkzeug.security.generate_password_hash()` produces the hash before this insert runs.

Query 2: Create a New Public Channel (with Authorization Check)

```

1 INSERT INTO channel (workspace_id, name, channel_type, creator_id)
2 SELECT 1, 'announcements', 'public', 2
3 WHERE EXISTS (
4     SELECT 1 FROM workspace_member
5     WHERE workspace_id = 1
6         AND user_id = 2
7         AND is_active = TRUE
8 );

```

Result: INSERT 0 1. The WHERE EXISTS subquery enforces that the requesting user is an active workspace member, without using database-level permissions. If the check fails, zero rows are inserted and no channel is created.

Query 3: List All Administrators per Workspace

```

1 SELECT w.name AS workspace_name, u.username, u.email
2 FROM workspace_admin wa
3 JOIN workspace w ON w.workspace_id = wa.workspace_id
4 JOIN users u ON u.user_id = wa.user_id
5 ORDER BY w.name, u.username;

```

Result:

workspace_name	username	email
Coop Board	alice	alice@nyu.edu
NYU Research	alice	alice@nyu.edu
NYU Research	bob	bob@nyu.edu

Query 4: Public Channel Invitations Older Than 5 Days With No Join

```

1 SELECT c.name AS channel_name, u.username, ci.invited_at
2 FROM channel_invitation ci
3 JOIN channel c ON c.channel_id = ci.channel_id
4 JOIN users u ON u.user_id = ci.invitee_id
5 WHERE c.channel_type = 'public'
6     AND ci.status = 'pending'
7     AND ci.invited_at < NOW() - INTERVAL '5 days'
8     AND NOT EXISTS (
9         SELECT 1 FROM channel_member cm
10        WHERE cm.channel_id = ci.channel_id
11            AND cm.user_id = ci.invitee_id
12 );

```

Result:

channel_name	username	invited_at
general	dave	2026-04-19 11:43:32

The NOT EXISTS subquery confirms the invitee has not joined `channel_member` independently. The sample data includes a second dave invitation two days old, which correctly does not appear.

Query 5: All Messages in a Channel, Chronological Order

```

1 SELECT u.username, m.body, m.posted_at
2 FROM message m
3 JOIN users u ON u.user_id = m.sender_id
4 WHERE m.channel_id = 1
5 ORDER BY m.posted_at ASC;

```

Result:

username	body	posted_at
alice	Welcome everyone to the research workspace!	2026-04-22
bob	Thanks Alice, excited to be here.	2026-04-23
carol	The vector must be perpendicular to the surface.	2026-04-24
alice	Great point Carol, noted.	2026-04-25

Query 6: All Messages Posted by a Particular User

```

1 SELECT c.name AS channel_name, m.body, m.posted_at
2 FROM message m
3 JOIN channel c ON c.channel_id = m.channel_id
4 WHERE m.sender_id = 1
5 ORDER BY m.posted_at ASC;

```

Result:

channel_name	body	posted_at
general	Welcome everyone to the research workspace!	2026-04-22
general	Great point Carol, noted.	2026-04-25
hiring	This channel is for hiring discussions only.	2026-04-26
announcements	Coop board meeting this Friday.	2026-04-26
alice-bob	Hey Bob, quick DM.	2026-04-27

Query 7: Accessible Messages Containing a Keyword

```

1 SELECT c.name AS channel_name, m.body, m.posted_at
2 FROM message m
3 JOIN channel c ON c.channel_id = m.channel_id
4 JOIN workspace_member wm ON wm.workspace_id = c.workspace_id
5     AND wm.user_id = 3
6 JOIN channel_member cm ON cm.channel_id = m.channel_id
7     AND cm.user_id = 3
8 WHERE m.body ILIKE '%perpendicular%'
9 ORDER BY m.posted_at ASC;

```

Result:

channel_name	body	posted_at
general	The vector must be perpendicular to the surface.	2026-04-24

The two JOINS on `workspace_member` and `channel_member` enforce access control: results are restricted to channels the requesting user is a member of. `ILIKE` provides case-

insensitive matching. This query is reused verbatim in the web application’s search feature.

Sample Data

The test dataset was designed to exercise all seven queries and cover edge cases including multi-admin workspaces, all three channel types, stale invitations at varying ages, and messages spanning multiple channels.

Entity	Instance	Key relationships
users	alice (1), bob (2), carol (3), dave (4), eve (5)	
workspace	NYU Research (1) Coop Board (2)	creator: alice; admins: alice, bob creator: alice; admin: alice
channel	general (1, public) hiring (2, private) alice-bob (3, direct) announcements (4, public) announcements (5, public)	workspace 1; members: alice, bob, carol workspace 1; members: alice, bob workspace 1; members: alice, bob workspace 2; members: alice, dave workspace 1; created by bob
message	7 messages	channel 1 has 4 msgs incl. “perpendicular”
invitations	dave invited to general (8 days ago, pending) dave invited to general (2 days ago, pending)	tests Query 4 tests 5-day boundary

Two admins in workspace 1 ensure Query 3 returns multiple rows. The second dave invitation tests that the 5-day threshold is applied correctly and boundary cases do not appear in Query 4 results. Eve exists as a registered user but holds no workspace membership, testing the invitation flow in isolation.

Assumptions

1. **Channel creation authorization.** Any active workspace member may create a channel. This is enforced via the `WHERE EXISTS` check in Query 2, consistent with the spec’s instruction not to use DBMS permissions.
2. **Direct channels.** Direct channels are workspace-scoped and created like any other channel. The two-user constraint is enforced at the application level rather than in the schema, since a schema-level constraint would require triggers.
3. **Workspace creator as admin.** The creator is explicitly inserted into `workspace_admin` by the application, not derived from `workspace.creator_id`. This makes `workspace_admin` the single source of truth for admin status.
4. **Channel membership is independent of workspace membership.** Joining a workspace does not automatically add a user to any channel. Channel membership requires an explicit join or accepted invitation.
5. **Password storage.** Passwords are stored as hashes using Werkzeug’s PBKDF2-HMAC-SHA256 implementation. Plaintext passwords are never persisted.
6. **No message deletion.** Consistent with the spec, no delete mechanism is implemented on the `message` table.

7. **Timestamps.** All timestamps use PostgreSQL's `TIMESTAMP` type without timezone, defaulting to server time via `NOW()`.

Project 2: Web Frontend

Overview

Project 2 builds a complete web application on top of the Project 1 schema. All operations available in the system — registration, login, workspace and channel management, messaging, invitation handling, and search — are accessible via a standard web browser. The application is deployed publicly at <https://snickr-production.up.railway.app>.

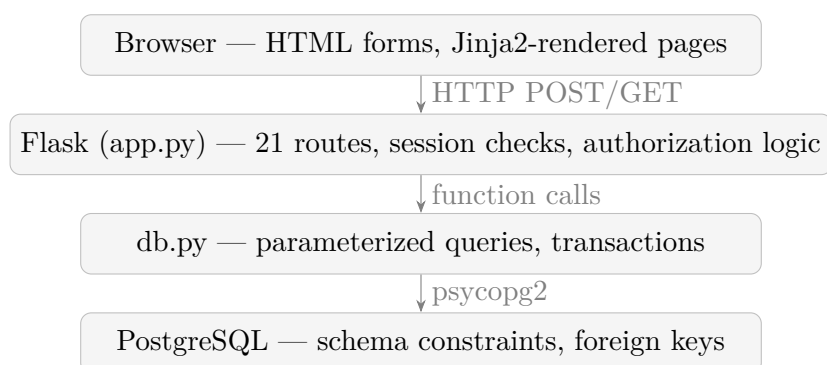
Technology Stack

Layer	Technology	Rationale
Language	Python 3	Available background; readable, concise
Web framework	Flask 3.0	Lightweight; routes map directly to functions
Database driver	psycopg2 2.9	Native parameterized queries; explicit transaction control
Templating	Jinja2	Bundled with Flask; auto-escapes all output by default
Password hashing	Werkzeug	PBKDF2-HMAC-SHA256; industry standard
Session management	Flask signed cookies	Cryptographically signed; tamper-evident
Database	PostgreSQL 18	CHECK constraints, SERIAL, ILIKE, transactions
Production server	Gunicorn 26	WSGI-compliant; required for Railway deployment

No ORM is used. All queries are written in raw SQL in `db.py` and called from `app.py`. This keeps the database layer explicit and auditable.

System Architecture

The application follows a three-tier architecture:



Project Structure

```

snickr/
  app.py          # All Flask routes (21 routes)
  db.py           # All database queries and transactions
  requirements.txt
  Procfile        # Gunicorn start command for Railway
  seed.sql        # Realistic sample data
  
```

```

schema.sql          # Full DDL for deployment
.env.example        # Environment variable template
static/
    style.css
templates/
    base.html
    invitations.html
    search.html
    auth/login.html, register.html
    channel/create.html, detail.html
    workspace/create.html, detail.html, list.html

```

Session Management

When a user logs in, Flask stores their `user_id` in a signed cookie using its built-in session mechanism. The cookie is cryptographically signed with the application's secret key using HMAC, so any modification by the client is detected and the session is invalidated. On every subsequent request, the application reads `session['user_id']` to identify the requesting user.

The database has a single connection user. There are no per-user DBMS accounts. The spec explicitly requires this model: the web application logs into the database as one user and restricts access at the application level based on the cookie's `user_id`.

Routes

Route	Method	Description
/	GET	Redirect to workspaces or login
/register	GET/POST	Registration form; hash password; set session cookie
/login	GET/POST	Login form; verify hash; set session cookie
/logout	GET	Clear session cookie
/workspaces	GET	List active workspaces for logged-in user
/workspaces/create	GET/POST	Create workspace (transactional)
/workspaces/<id>	GET	Workspace detail: channels sidebar, members panel
/workspaces/<id>/invite	POST	Admin-only workspace invitation
/workspaces/<id>/grant-admin	POST	Admin-only: promote member to admin
/workspaces/<id>/channels/create	GET/POST	Create channel (workspace member only)
/channels/<id>	GET	Channel view with messages
/channels/<id>/join	POST	Join public channel directly
/channels/<id>/post	POST	Post message
/channels/<id>/invite	POST	Invite user to channel
/invitations	GET	View all pending invitations
/invitations/<id>/respond	POST	Accept or decline workspace invitation
/channel-invitations/<id>/respond	POST	Accept or decline channel invitation
/search	GET	Keyword search (Query 7 logic)

Security

SQL Injection Prevention

SQL injection occurs when user input is concatenated into a query string, allowing malicious input to modify the query's structure. The canonical example is entering `' OR '1'='1` as a username to bypass authentication.

Snickr prevents this by using `psycopg2` parameterized queries throughout `db.py`. Every query uses `%s` placeholders; the SQL string and the parameter values are sent to PostgreSQL as separate arguments. PostgreSQL treats the parameter as a literal string value and never interprets it as SQL syntax. There is no string concatenation of user input anywhere in the codebase.

```
# Safe -- parameterized
cur.execute("SELECT * FROM users WHERE username = %s", (username,))

# Unsafe -- never done in this application
cur.execute("SELECT * FROM users WHERE username = '" + username + "'")
```

Cross-Site Scripting Prevention

XSS occurs when user-supplied content is rendered as raw HTML in the browser, allowing injected `<script>` tags to execute in other users' sessions. Jinja2 auto-escapes all variables rendered with the `{{ }}` syntax by default, converting `<` to `<`, `>` to `>`, and so on. No `|safe` filters are applied to user-generated content anywhere in the templates.

Password Security

Passwords are hashed with Werkzeug's `generate_password_hash()`, which uses PBKDF2-HMAC-SHA256 with a random salt. The hash is stored in the `password_hash` column; the plaintext password is never persisted. Login uses `check_password_hash(stored_hash, input_password)`, which hashes the input and compares the results without ever comparing plaintexts. Even if the database were fully compromised, stored hashes cannot be reversed to recover plaintext passwords.

Authorization

Every route that accesses workspace or channel content calls `is_workspace_member()` or `is_channel_member()` before serving any data. Unauthenticated users are redirected by the `@login_required` decorator before any route logic runs. Admin-only actions (workspace invitation, grant-admin) additionally call `is_workspace_admin()`.

Access control is enforced entirely at the application level, as the spec requires. The database connection user (`postgres`) can read all tables, but the application ensures each request only accesses data the session user is permitted to see.

Transactions

The database connection uses `autocommit = False`. All write operations are committed explicitly; failures trigger a rollback via `try/except`. Three operations require multi-

statement atomicity.

Create Workspace

Creating a workspace requires three inserts that must all succeed or all fail:

1. `INSERT INTO workspace` → obtain `workspace_id`
2. `INSERT INTO workspace_member` (creator becomes member)
3. `INSERT INTO workspace_admin` (creator becomes admin)

Without a transaction, a crash between steps 2 and 3 would produce a workspace with no admin, leaving the system in an inconsistent state that no route could recover.

Accept Workspace Invitation

Accepting a workspace invitation requires:

1. `UPDATE workspace_invitation SET status = 'accepted'`
2. `INSERT INTO workspace_member`

Without atomicity, a crash between these two statements would leave the invitation marked accepted but the user absent from `workspace_member`, granting them no actual access.

Accept Channel Invitation

The same pattern applies to channel invitations:

1. `UPDATE channel_invitation SET status = 'accepted'`
2. `INSERT INTO channel_member`

All three transactions use the same implementation pattern: a `try` block containing the full sequence of cursor executions, followed by `conn.commit()` on success and `conn.rollback()` in the `except` clause.

Concurrency

PostgreSQL's MVCC (Multi-Version Concurrency Control) system handles concurrent access without explicit locking for most operations. Multiple users can post messages, join channels, and view workspaces simultaneously. The unique constraints on invitation tables (`workspace_invitation(workspace_id, invitee_id)` and `channel_invitation(channel_id, invitee_id)`) prevent duplicate invitation rows even if two admins attempt to invite the same user concurrently; the second insert hits `ON CONFLICT DO NOTHING` and is silently discarded.

Application Walkthrough

The following describes a complete user session demonstrating all major features. The session was conducted against the live deployment.

1. Navigate to `/register` → create account with email, username, password
- ↓
2. Redirect to `/workspaces` → session cookie set; workspace list shown
- ↓
3. Create workspace “Axiom HQ” → transaction inserts workspace, member, admin rows
- ↓
4. Create public channel `#general` → authorization check passes; creator added as member
- ↓
5. Create private channel `#hiring` → invite-only; invite form visible to creator
- ↓
6. Invite second user to workspace → invitation row inserted with status `pending`
- ↓
7. Second user logs in, visits `/invitations` → pending invitation shown
- ↓
8. Second user accepts → transaction: status updated, `workspace_member` row inserted
- ↓
9. First user promotes second user to admin → `workspace_admin` row inserted
- ↓
10. Post messages in `#general` → messages stored with `sender_id` and timestamp
- ↓
11. Search for keyword → Query 7 logic returns only accessible results

At each step the relevant database tables are updated: `users` at registration, `workspace` and junction tables at workspace creation, `channel` and `channel_member` at channel creation, `workspace_invitation` and `workspace_member` through the invitation flow, `workspace_admin` at the grant-admin step, `message` at each post, and the full search query executed at step 11.

URL state is preserved throughout: workspace URLs follow `/workspaces/{id}`, channel URLs follow `/channels/{id}`, and search results carry the keyword as a query parameter (`/search?q=keyword`), making them bookmarkable as noted in the spec.

Additional Features

Beyond the baseline requirements, the following features were implemented.

Admin promotion. Workspace admins can promote any active workspace member to admin via the `/workspaces/{id}/grant-admin` route. The route validates that the requesting user is an admin, the target user exists and is a workspace member, and the target is not already an admin. The grant is a single `INSERT` into `workspace_admin` with `ON CONFLICT DO NOTHING`.

Bookmarkable URLs. Workspaces, channels, and search results all have stable, meaningful URLs that users can bookmark and share. The search route uses HTTP GET with a `q` query parameter, meaning search results are directly linkable.

Environment-based configuration. Database credentials and the Flask secret key are read from environment variables via `python-dotenv`, keeping secrets out of the codebase. A `.env.example` file documents the required variables. `.env` is excluded from version

control via `.gitignore`.

Public deployment. The application is deployed on Railway at <https://snickr-production.up.railway.app> with a Railway-managed PostgreSQL instance. The production server is Gunicorn rather than Flask's development server. The deployment uses the same schema and application code as the local development environment.

Deployment

The application runs on Railway (<https://railway.app>) with two services: a Gunicorn-served Flask application and a Railway-managed PostgreSQL 16 instance. The application connects to PostgreSQL over Railway's internal network using the `postgres.railway.internal` hostname.

The `Procfile` specifies the start command:

```
web: gunicorn app:app --bind 0.0.0.0:$PORT
```

Railway injects the `PORT` environment variable at runtime. The application reads it via `os.environ.get("PORT", 5000)` and passes it to Gunicorn. Database credentials are set as Railway environment variables and read via `python-dotenv` at startup.

The schema was loaded by running `schema.sql` against the Railway PostgreSQL public URL from a local terminal. Realistic sample data covering two workspaces, seven channels, and fifty messages was loaded from `seed.sql`. Source code is available at <https://github.com/shyamksateesh/snickr>.